COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-CPE21S4	**SECTION:**	CPE21S4
DATE PERFORMED:	23/09/2026	**DATE SUBMITTED:**	23//09/2026
NAME:	Jullian Edrain R. Sevilla	**INSTRUCTOR:**	Engr. JIMLORD QUEJADO

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

main.cpp

tree > main.cpp > ...

```cpp
#include <iostream>
#include <vector>
using namespace std;

struct GNode {
    char data;
    vector<GNode*> children;
    GNode(char val) {
        data = val;
    }
};

GNode* buildTree() {
    GNode* A = new GNode('A');
    GNode* B = new GNode('B');
    GNode* C = new GNode('C');
    GNode* D = new GNode('D');
    GNode* E = new GNode('E');
    GNode* F = new GNode('F');
    GNode* G = new GNode('G');

    A->children.push_back(B);
    A->children.push_back(C);
    A->children.push_back(D);
    A->children.push_back(E);
    A->children.push_back(F);
    A->children.push_back(G);

    GNode* H = new GNode('H');
    D->children.push_back(H);

    GNode* I = new GNode('I');
    GNode* J = new GNode('J');
    E->children.push_back(I);
    E->children.push_back(J);

    GNode* K = new GNode('K');
    GNode* L = new GNode('L');
    GNode* M = new GNode('M');
    F->children.push_back(K);
    F->children.push_back(L);
    F->children.push_back(M);

    GNode* N = new GNode('N');
    G->children.push_back(N);
```

main.cpp

tree > main.cpp > buildTree()

```cpp
GNode* buildTree() {
    GNode* Q = new GNode('Q');
    J->children.push_back(P);
    J->children.push_back(Q);

    return A;
}

void preOrder(GNode* n) {
    if (n == NULL) return;
    cout << n->data << " ";
    for (int i = 0; i < n->children.size(); i++) {
        preOrder(n->children[i]);
    }
}

void postOrder(GNode* n) {
    if (n == NULL) return;
    for (int i = 0; i < n->children.size(); i++) {
        postOrder(n->children[i]);
    }
    cout << n->data << " ";
}

int main() {
    GNode* root = buildTree();

    cout << "Pre-order:  ";
    preOrder(root);
    cout << "\n";

    cout << "Post-order: ";
    postOrder(root);
    cout << "\n";

    return 0;
}
```

```
[Running] cd "c:\Users\Jullian Sevilla\OneDrive\Documents\DSA\Midterms\9.1\tree\" && g++ main.cpp -o main && "c:\Users\Jullian Sevilla\OneDrive\Documents\DSA\Midterms\9.
1\tree\"main
Pre-order:  A B C D H E I J P Q F K L M G N
Post-order: B C H D I P Q J E K L M F N G A

[Done] exited with code=0 in 0.758 seconds
```

ILOB.cpp

tree > ILOB.cpp > main()

```cpp
#include <iostream>
using namespace std;

struct BNode {
    int data;
    BNode* left;
    BNode* right;
    BNode(int val) {
        data = val;
        left = NULL;
        right = NULL;
    }
};

BNode* buildTree() {
    BNode* root = new BNode(27);
    root->left = new BNode(14);
    root->left->left = new BNode(10);
    root->left->right = new BNode(19);
    root->right = new BNode(35);
    root->right->left = new BNode(31);
    root->right->right = new BNode(42);
    return root;
}

void preOrder(BNode* n) {
    if (n == NULL) return;
    cout << n->data << " ";
    preOrder(n->left);
    preOrder(n->right);
}

void inOrder(BNode* n) {
    if (n == NULL) return;
    inOrder(n->left);
    cout << n->data << " ";
    inOrder(n->right);
}

void postOrder(BNode* n) {
    if (n == NULL) return;
    postOrder(n->left);
    postOrder(n->right);
    cout << n->data << " ";
}
```

ILOB.cpp

tree > ILOB.cpp > ...

```cpp
void findData(BNode* n, int choice, int key) {
    if (n == NULL) return;
    if (n->data == key) {
        cout << key << " was found!\n";
        found = true;
        return;
    }
    if (choice == 1) {
        findData(n->left, choice, key);
        if (!found) findData(n->right, choice, key);
    }
    else if (choice == 2) {
        findData(n->left, choice, key);
        if (!found) findData(n->right, choice, key);
    }
    else if (choice == 3) {
        findData(n->left, choice, key);
        findData(n->right, choice, key);
    }
}

int main() {
    BNode* root = buildTree();

    cout << "Pre-order:  ";
    preOrder(root);
    cout << "\n";

    cout << "In-order:   ";
    inOrder(root);
    cout << "\n";

    cout << "Post-order: ";
    postOrder(root);
    cout << "\n\n";

    cout << "Find 19 using Pre-order:\n";
    found = false;
    findData(root, 1, 19);

    cout << "\nFind 31 using In-order:\n";
    found = false;
    findData(root, 2, 31);

    cout << "\nFind 42 using Post-order:\n";
```

```
93      cout << "\nFind 42 using Post-order:\n";
94      found = false;
95      findData(root, 3, 42);
96
97      return 0;
98  }
```

```
[Running] cd "c:\Users\Jullian Sevilla\OneDrive\Dc
1\tree\"ILOB
Pre-order:  27 14 10 19 35 31 42
In-order:   10 14 19 27 31 35 42
Post-order: 10 19 14 31 42 35 27

Find 19 using Pre-order:
19 was found!

Find 31 using In-order:
31 was found!

Find 42 using Post-order:
42 was found!

[Done] exited with code=0 in 0.751 seconds
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILOB.cpp ILOC.cpp

tree > ILOC.cpp > ...

```cpp
#include <iostream>
using namespace std;

struct BNode {
    int data;
    BNode* left;
    BNode* right;
    BNode(int val) {
        data = val;
        left = NULL;
        right = NULL;
    }
};

BNode* insert(BNode* n, int val) {
    if (n == NULL) {
        return new BNode(val);
    }
    if (val < n->data) {
        n->left = insert(n->left, val);
    }
    else if (val > n->data) {
        n->right = insert(n->right, val);
    }
    return n;
}

void preOrder(BNode* n) {
    if (n == NULL) return;
    cout << n->data << " ";
    preOrder(n->left);
    preOrder(n->right);
}

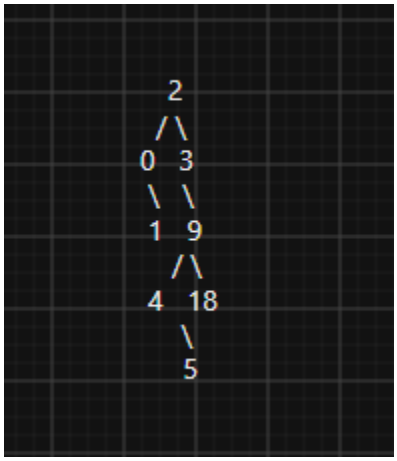
void inOrder(BNode* n) {
    if (n == NULL) return;
    inOrder(n->left);
    cout << n->data << " ";
    inOrder(n->right);
}

void postOrder(BNode* n) {
    if (n == NULL) return;
    postOrder(n->left);
    postOrder(n->right);
    cout << n->data << " ";
```

```
48
49  int main() {
50      BNode* root = NULL;
51      int values[] = {2, 3, 9, 18, 0, 1, 4, 5};
52
53      for (int i = 0; i < 8; i++) {
54          root = insert(root, values[i]);
55      }
56
57      cout << "Pre-order:  ";
58      preOrder(root);
59      cout << "\n";
60
61      cout << "In-order:   ";
62      inOrder(root);
63      cout << "\n";
64
65      cout << "Post-order: ";
66      postOrder(root);
67      cout << "\n";
68
69      return 0;
70  }
```

```
[Running] cd "c:\Users\Jullian Sevilla\OneDrive\Doc
1\tree\"ILOC
Pre-order:  2 0 1 3 9 4 5 18
In-order:   0 1 2 3 4 5 9 18
Post-order: 1 0 5 4 18 9 3 2

[Done] exited with code=0 in 0.714 seconds
```



```cpp
void preOrder(BNode* n) {
    if (n == NULL) return;
    cout << n->data << " ";
    preOrder(n->left);
    preOrder(n->right);
}

void inOrder(BNode* n) {
    if (n == NULL) return;
    inOrder(n->left);
    cout << n->data << " ";
    inOrder(n->right);
}

void postOrder(BNode* n) {
    if (n == NULL) return;
    postOrder(n->left);
    postOrder(n->right);
    cout << n->data << " ";
}
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.